

NAME - PRERANA MUDIAR
ROLL NO - CSB24021

```
import java.util.*;
import java.util.stream.*;

class Student {
    private int id;
    private String name;
    private List<String> courses;
    private Map<String, Integer> scores;

    public Student(int id, String name, List<String> courses, Map<String, Integer> scores) {
        this.id = id;
        this.name = name;
        this.courses = courses;
        this.scores = scores;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public List<String> getCourses() {
        return courses;
    }

    public Map<String, Integer> getScores() {
        return scores;
    }

    // Calculate average score of student
    public double getAverageScore() {
        return courses.stream()
            .mapToInt(course -> scores.getDefault(course, 0))
            .average()
            .orElse(0.0);
    }

    @Override
    public String toString() {
        return "ID: " + id + ", Name: " + name + ", Avg: " + getAverageScore();
    }
}
```

```

public class StudentPerformanceAnalyzer {

    // Get Top N Students (sorted by average descending)
    public static List<Student> getTopNStudents(List<Student> students, int n) {
        return students.stream()
            .sorted(Comparator.comparingDouble(Student::getAverageScore).reversed())
            .limit(n)
            .collect(Collectors.toList());
    }

    // Get Average Score Per Course
    public static Map<String, Double> getAverageScorePerCourse(List<Student> students) {

        Set<String> allCourses = getAllUniqueCourses(students);

        Map<String, Double> courseAverage = new HashMap<>();

        for (String course : allCourses) {
            double avg = students.stream()
                .mapToInt(student -> student.getScores().getOrDefault(course, 0))
                .average()
                .orElse(0.0);

            courseAverage.put(course, avg);
        }

        return courseAverage;
    }

    // Get All Unique Courses
    public static Set<String> getAllUniqueCourses(List<Student> students) {
        return students.stream()
            .flatMap(student -> student.getCourses().stream())
            .collect(Collectors.toCollection(HashSet::new));
    }

    // Main Method
    public static void main(String[] args) {

        List<Student> students = new ArrayList<>();

        // Student 1
        Map<String, Integer> scores1 = new HashMap<>();
        scores1.put("Math", 90);
        scores1.put("Physics", 85);
        students.add(new Student(1, "Alice",
            Arrays.asList("Math", "Physics"), scores1));
    }
}

```

```

// Student 2
Map<String, Integer> scores2 = new HashMap<>();
scores2.put("Math", 75);
scores2.put("Chemistry", 80);
students.add(new Student(2, "Bob",
    Arrays.asList("Math", "Chemistry"), scores2));

// Student 3
Map<String, Integer> scores3 = new HashMap<>();
scores3.put("Physics", 95);
scores3.put("Chemistry", 88);
students.add(new Student(3, "Charlie",
    Arrays.asList("Physics", "Chemistry"), scores3));

System.out.println("Top 2 Students:");
getTopNStudents(students, 2).forEach(System.out::println);

System.out.println("\nAverage Score Per Course:");
getAverageScorePerCourse(students)
    .forEach((course, avg) ->
        System.out.println(course + " : " + avg));

System.out.println("\nAll Unique Courses:");
getAllUniqueCourses(students).forEach(System.out::println);
}
}

```

The screenshot shows an IDE window with the following components:

- Explorer:** Shows the project structure with 'StudentPerformanceAnalyzer.class' selected.
- Output:** Displays the program's output, which includes the top 2 students (Charlie and Alice), the average score per course (Chemistry: 56.0, Math: 55.0, Physics: 60.0), and all unique courses (Chemistry, Math, Physics).
- Status Bar:** Indicates the program exited successfully with code=0 in 1.586 seconds.